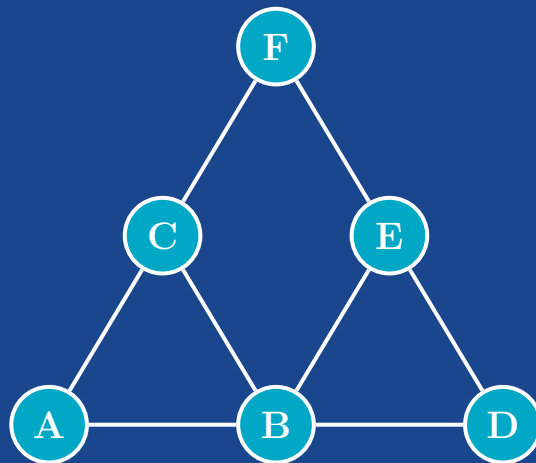




Graph Algorithms

Complete Reference Guide

Data Structures & Algorithms — Chapter 13

BFS · DFS · Topological Sort · Shortest Paths · MST · Advanced Algorithms

Problems Covered:	42
Categories:	6
Algorithms:	Dijkstra, Bellman-Ford, Floyd-Warshall, Kruskal, Prim, Tarjan, Kosaraju

Contents

1	Overview & Problem Index	3
2	Graph Categories — Mind Map	5
3	Category 1 — Fundamentals	6
3.1	Problem 1 — Count the Number of Graphs	6
3.2	Problem 2 — Graph Representation (Matrix $\rightarrow$ Adjacency List)	6
4	Category 2 — Traversal Problems	8
4.1	Problem 3 — BFS (Breadth-First Search)	8
4.2	Problem 4 — DFS (Depth-First Search)	9
4.3	Problem 5 — Count the Number of Provinces	10
4.4	Problem 6 — Rotten Oranges	10
4.5	Problem 7 — Flood Fill Algorithm	11
4.6	Problem 8 — Detect Cycle in Undirected Graph	11
4.7	Problem 9 — 01 Matrix (Distance to Nearest Zero)	12
4.8	Problem 10 — Surrounded Regions	12
4.9	Problem 11 — Number of Enclaves	12
4.10	Problem 12 — Word Ladder	13
4.11	Problem 13 — Distinct Islands	13
4.12	Problem 14 — Bipartite Graph	14
4.13	Problem 15 — Detect Cycle in Directed Graph	14
5	Category 3 — Topological Sort Problems	15
5.1	Problem 16 — Topological Sorting (DFS)	15
5.2	Problem 17 — Kahn's Algorithm (BFS Topo Sort)	16
5.3	Problem 18 — Course Scheduler 1	16
5.4	Problem 19 — Course Scheduler 2	17
5.5	Problem 20 — Find Eventual Safe States	17
5.6	Problem 21 — Alien Dictionary	17
6	Category 4 — Shortest Path Problems	19
6.1	Problem 22 — Shortest Path (Unit Distance)	19
6.2	Problem 23 — Shortest Path in DAG	19
6.3	Problem 24 — Dijkstra's Algorithm	20
6.4	Problem 25 — Shortest Path in Binary Matrix	21
6.5	Problem 26 — Path with Minimum Effort	21
6.6	Problem 27 — Cheapest Flights within K Stops	21
6.7	Problem 28 — Network Delay Time	22
6.8	Problem 29 — Bellman-Ford Algorithm	22
6.9	Problem 30 — Floyd-Warshall Algorithm	23
6.10	Problem 31 — City with Smallest Number of Neighbours	23
6.11	Problem 32 — Number of Ways to Arrive at Destination	23

7	Category 5 — Minimum Spanning Tree & Disjoint Set Union	25
7.1	Problem 33 — Prim's Algorithm	25
7.2	Problem 34 — Kruskal's Algorithm	25
7.3	Problem 35 — Number of Operations to Make Network Connected	26
7.4	Problem 36 — Most Stones Removed	26
7.5	Problem 37 — Account Merge	27
7.6	Problem 38 — Number of Islands 2 (Online DSU)	27
7.7	Problem 39 — Making Large Island	27
7.8	Problem 40 — Swim in Rising Water	28
8	Category 6 — Advanced Graph Algorithms	29
8.1	Problem 41 — Bridges in a Graph (Tarjan's Algorithm)	29
8.2	Problem 42 — Strongly Connected Components (Kosaraju's Algorithm)	30
9	Complete Complexity Summary Table	32
10	Algorithm Patterns — Quick Reference	34
11	Algorithm Decision Tree	36

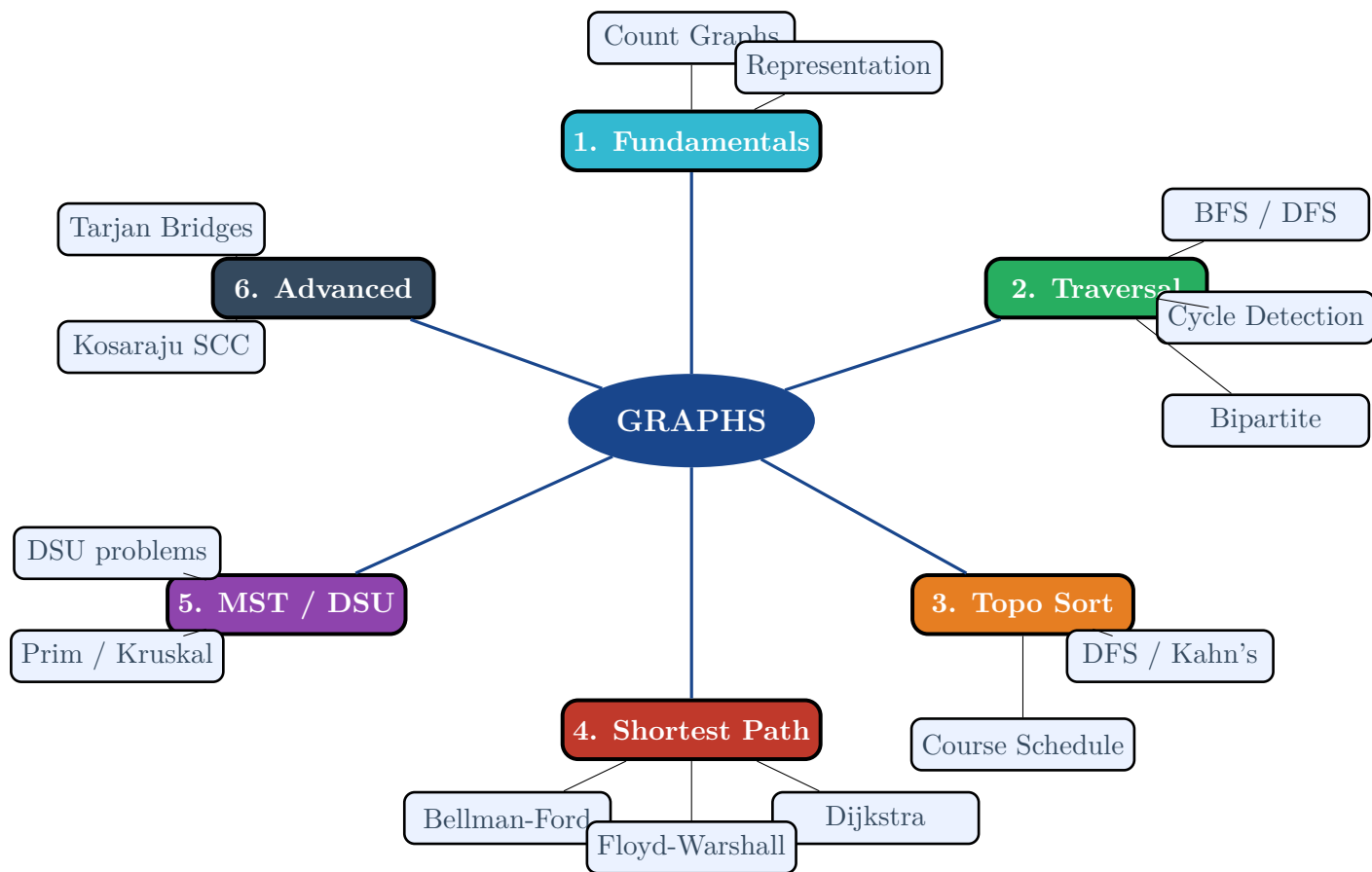
§1 Overview & Problem Index

This reference covers every graph problem from Chapter 13, organised into six categories. Each problem includes a description, step-by-step approach, complexity analysis, and visualisation where applicable.

#	Problem	Category	Time	Space
1	Count Graphs	Fundamentals	$O(1)$	$O(1)$
2	Graph Representation	Fundamentals	$O(m)$	$O(n+2m)$
3	BFS	Traversal	$O(V+E)$	$O(V)$
4	DFS	Traversal	$O(V+E)$	$O(V)$
5	Count Provinces	Traversal	$O(n^2)$	$O(n)$
6	Rotten Oranges	Traversal	$O(m*n)$	$O(m*n)$
7	Flood Fill	Traversal	$O(m*n)$	$O(m*n)$
8	Cycle in Undirected	Traversal	$O(V+E)$	$O(V)$
9	01 Matrix	Traversal	$O(m*n)$	$O(m*n)$
10	Surrounded Regions	Traversal	$O(m*n)$	$O(m*n)$
11	Number of Enclaves	Traversal	$O(m*n)$	$O(m*n)$
12	Word Ladder	Traversal	$O(n^2 * m)$	$O(n^2)$
13	Distinct Islands	Traversal	$O(n*m)$	$O(n*m)$
14	Bipartite Graph	Traversal	$O(V+E)$	$O(V)$
15	Cycle in Directed	Traversal	$O(V+E)$	$O(V)$
16	Topological Sort	Topo Sort	$O(V+E)$	$O(V)$
17	Kahn's Algorithm	Topo Sort	$O(V+E)$	$O(V)$
18	Course Scheduler 1	Topo Sort	$O(N+E)$	$O(N+E)$
19	Course Scheduler 2	Topo Sort	$O(N+E)$	$O(N+E)$
20	Find Eventual Safe	Topo Sort	$O(N+E)$	$O(N+E)$
21	Alien Dictionary	Topo Sort	$O(N)$	$O(K)$
22	Shortest Path Unit	Shortest Path	$O(N+M)$	$O(N)$
23	Shortest Path DAG	Shortest Path	$O(N+M)$	$O(N+M)$
24	Dijkstra's Algorithm	Shortest Path	$O(E \log V)$	$O(V+E)$
25	Shortest Path Binary	Shortest Path	$O(n^2)$	$O(n^2)$
26	Min Effort Path	Shortest Path	$O(mn \log mn)$	$O(mn)$
27	Cheapest K Stops	Shortest Path	$O(E*K)$	$O(V)$
28	Network Delay Time	Shortest Path	$O(E \log V)$	$O(V+E)$
29	Bellman-Ford	Shortest Path	$O(V*E)$	$O(V)$
30	Floyd-Warshall	Shortest Path	$O(V^3)$	$O(V^2)$
31	City Min Neighbours	Shortest Path	$O(V^3)$	$O(V^2)$
32	Ways to Destination	Shortest Path	$O(E \log V)$	$O(V)$
33	Prim's Algorithm	MST	$O(E \log V)$	$O(V+E)$
34	Kruskal's Algorithm	MST	$O(E \log E)$	$O(V)$
35	Number of Operations	MST/DSU	$O(n*\alpha)$	$O(n)$
36	Most Stones Removed	MST/DSU	$O(n*\alpha)$	$O(n)$
37	Account Merge	MST/DSU	$O(n \log n)$	$O(n)$
38	Number of Islands 2	MST/DSU	$O(k*\alpha)$	$O(n*m)$
39	Making Large Island	MST/DSU	$O(n^2)$	$O(n^2)$
40	Swim in Rising Water	MST	$O(n^2 \log n)$	$O(n^2)$

#	Problem	Category	Time	Space
41	Bridges in Graph	Advanced	$O(V+E)$	$O(V)$
42	Strongly Connected	Advanced	$O(V+E)$	$O(V)$

§2 Graph Categories — Mind Map



§3 Category 1 — Fundamentals

Category 1: Fundamentals

These problems build the foundation of graph theory: counting possible graphs from n vertices, and converting between matrix and list representations.

3.1 Problem 1 — Count the Number of Graphs

P1: Count the Number of Graphs

Given n vertices, count the total number of **distinct undirected graphs** that can be formed.

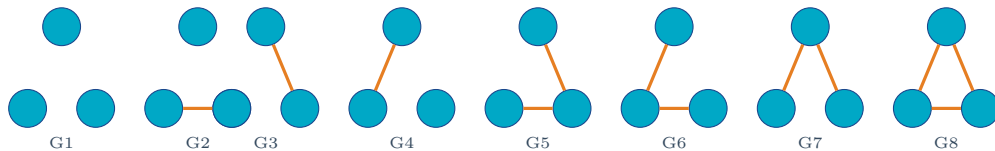
Approach

1. For n vertices, the maximum number of edges in an undirected graph is:

$$E_{\max} = \frac{n(n-1)}{2}$$

2. Each edge can either be *present* or *absent*, giving 2 choices per edge.
3. Total graphs = $2^{E_{\max}}$

Visualisation



All $2^3 = 8$ graphs for $n = 3$

Time Complexity: $O(1)$

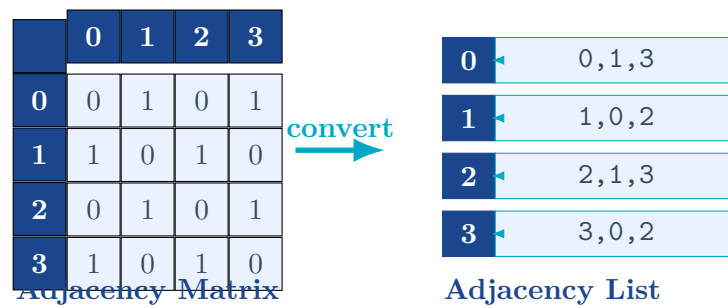
Space Complexity: $O(1)$

3.2 Problem 2 — Graph Representation (Matrix → Adjacency List)

P2: Graph Representation

Given an adjacency matrix of an undirected graph, return the adjacency list where each list starts with the vertex itself followed by its neighbours.

Matrix vs. Adjacency List



Approach

1. Initialise an empty `vector<vector<int>>` of size n .
2. For each vertex i , push i itself as the first element of `adj[i]`.
3. For every edge (u, v) in the matrix: push v into `adj[u]` and u into `adj[v]`.
4. Return the list.

Time Complexity: $O(m)$, m =edges

Space Complexity: $O(n + 2m)$

§4 Category 2 — Traversal Problems

Category 2: Traversal Problems

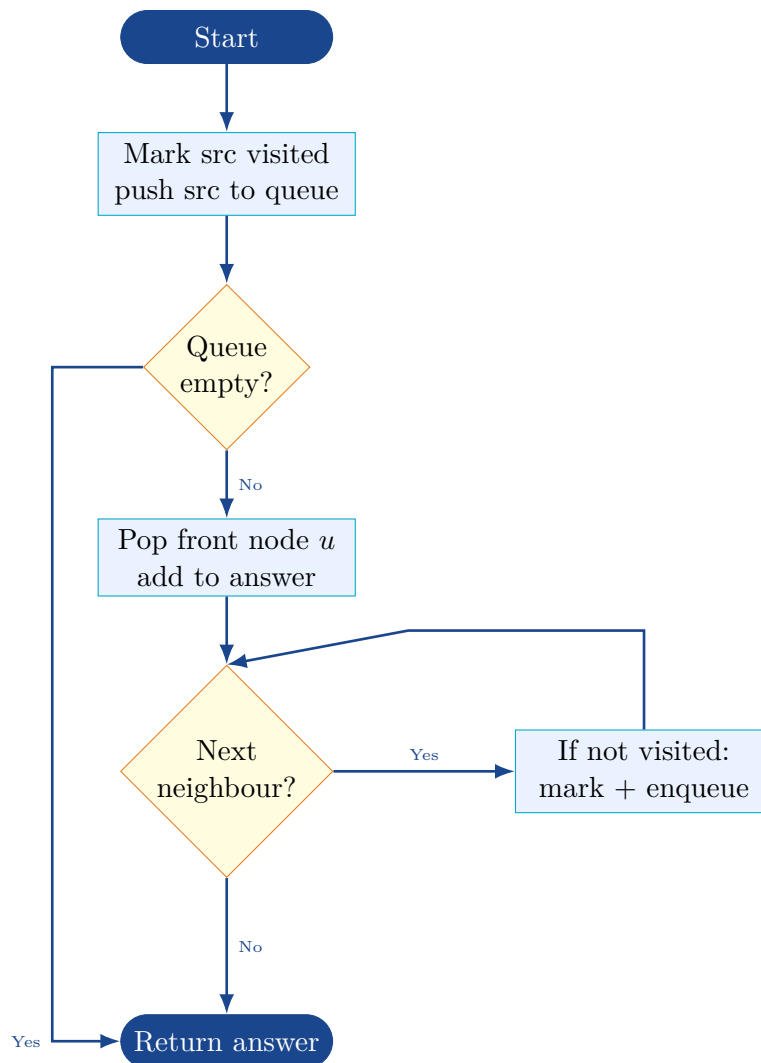
Problems solved using BFS and/or DFS as the core technique. Covers cycles, flood-fill, multi-source BFS, bipartite checking, word transformation and island problems.

4.1 Problem 3 — BFS (Breadth-First Search)

P3: BFS Traversal

Perform BFS on a directed graph starting from vertex 0 and return the traversal order.

BFS Algorithm Flowchart



Approach

1. Initialise **ans** vector and **vis[]** array set to false.
2. Push vertex 0 into a queue; mark it visited.
3. While queue is non-empty: pop front $\rightarrow$ add to **ans**; for each unvisited neighbour, mark visited and enqueue.
4. Return **ans**.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

4.2 Problem 4 — DFS (Depth-First Search)

P4: DFS Traversal

Perform DFS on a connected undirected graph from vertex 0.

BFS vs DFS Comparison

Property	BFS	DFS
Data Structure	Queue (FIFO)	Stack / Recursion
Traversal Order	Level by level	Branch-by-branch
Shortest path?	Yes (unweighted)	No
Memory	More (wide graphs)	Less (narrow graphs)
Cycle Detection	Yes	Yes
Typical Use	Shortest path, levels	Topological sort, SCCs

DFS Approach

1. Mark current node visited; push to **ans**.
2. Recursively call **dfs** on each unvisited neighbour.
3. All nodes reachable from source are visited by end.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$ stack

4.3 Problem 5 — Count the Number of Provinces

P5: Count Provinces

Given $n \times n$ matrix `isConnected`, find total number of *provinces* (connected components).

Approach

1. Convert matrix to adjacency list.
2. Iterate over all cities; for each unvisited city run DFS and increment province counter by 1.
3. DFS marks all cities in the same province as visited.

Time Complexity: $O(n^2)$

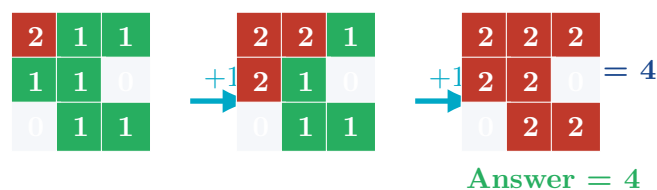
Space Complexity: $O(n)$

4.4 Problem 6 — Rotten Oranges

P6: Rotten Oranges (Multi-Source BFS)

Grid of 0=empty, 1=fresh, 2=rotten. Each minute, rotten infects adjacent fresh. Return min minutes or -1.

Multi-Source BFS Visualisation



Approach

1. Enqueue all initially rotten oranges; count fresh oranges.
2. BFS level by level (each level = 1 minute).
3. For each rotten orange, infect 4-directional fresh neighbours; decrement fresh count; enqueue new rotten oranges.
4. After BFS: if fresh count = 0 return time, else return -1.

Time Complexity: $O(m \cdot n)$

Space Complexity: $O(m \cdot n)$

4.5 Problem 7 — Flood Fill Algorithm

P7: Flood Fill

Starting from pixel (sr, sc) , replace all 4-directionally connected pixels of the same colour with `color`.

Approach

1. Record `oldColor = image[sr][sc]`.
2. If `oldColor == color`, return immediately (no change needed).
3. Run DFS/BFS from (sr, sc) ; paint each visited cell with `color` if its colour matches `oldColor`.

Time Complexity: $O(m \cdot n)$

Space Complexity: $O(m \cdot n)$ rec. stack

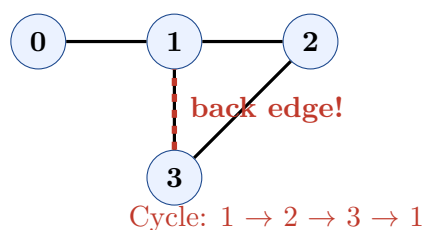
4.6 Problem 8 — Detect Cycle in Undirected Graph

P8: Cycle Detection — Undirected Graph

Given an undirected graph, determine whether it contains any cycle.

Key Idea

Back Edge Rule: In an undirected graph, a cycle exists if during DFS we visit a node that is already visited *and* it is not the immediate parent of the current node.



Approach

1. Run DFS from each unvisited node; track current node and its parent.
2. If a neighbour is visited and is *not* the parent $\Rightarrow$ cycle found.
3. Repeat for all disconnected components.

Time Complexity: $O(V + E)$
Space Complexity: $O(V)$

4.7 Problem 9 — 01 Matrix (Distance to Nearest Zero)

P9: 01 Matrix

For each cell in a binary matrix, find the distance to the nearest 0.

Approach — Multi-Source BFS from all zeros

1. Initialise distance matrix with -1 ; enqueue all 0 cells with distance 0.
2. BFS outwards: for each dequeued cell (r, c) with distance d , check 4 neighbours; if unvisited, set distance $= d + 1$ and enqueue.
3. This naturally propagates minimum distances from all zero sources.

Time Complexity: $O(m \cdot n)$
Space Complexity: $O(m \cdot n)$

4.8 Problem 10 — Surrounded Regions

P10: Surrounded Regions

Flip all 0s surrounded by X on all four sides to X. Border-connected 0s are safe.

Approach — Inverse DFS from borders

1. DFS from all 0s on the border; mark them safe (#).
2. Flip remaining 0s to X (they are surrounded).
3. Restore # back to 0.

Time Complexity: $O(m \cdot n)$
Space Complexity: $O(m \cdot n)$

4.9 Problem 11 — Number of Enclaves

P11: Number of Enclaves

Count land cells (1) from which you *cannot* walk off the grid.

Approach

1. DFS from all land cells on the boundary; mark them as visited.
2. Count remaining unvisited land cells — these are enclaves.

Time Complexity: $O(m \cdot n)$

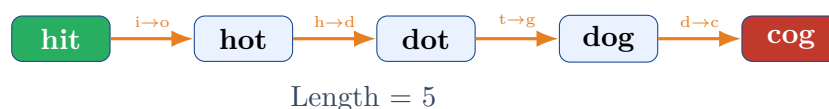
Space Complexity: $O(m \cdot n)$

4.10 Problem 12 — Word Ladder

P12: Word Ladder

Find the length of the shortest transformation chain from `beginWord` to `endWord` changing one letter at a time; each intermediate word must be in `wordList`.

BFS on Word Graph



Approach

1. Load `wordList` into a set for $O(1)$ lookup.
2. BFS from `beginWord`; for each word in queue, try replacing each character with `a-z`; if new word is in the set, enqueue it and remove it from set.
3. Return level count when `endWord` is reached.

Time Complexity: $O(n^2 \cdot m)$

Space Complexity: $O(n^2)$

4.11 Problem 13 — Distinct Islands

P13: Distinct Islands

Count islands that are *distinct* in shape (no rotation/reflection).

Approach — DFS Path Signature

1. DFS each unvisited land cell; record direction taken (U,D,L,R) and a B (backtrack) marker.
2. Store each island's path string in a `set<string>`.

3. Return the set size.

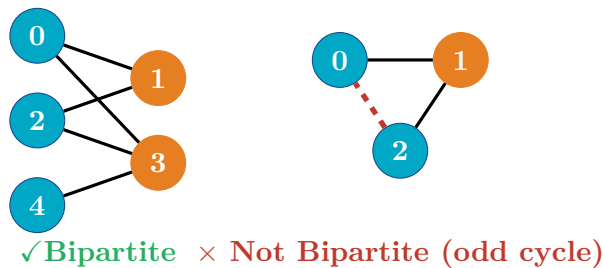
Time Complexity: $O(n \cdot m)$

Space Complexity: $O(n \cdot m)$

4.12 Problem 14 — Bipartite Graph

P14: Bipartite Check

Determine if a graph can be 2-coloured such that no two adjacent nodes share the same colour.



Approach

1. Colour source node with +1; run DFS/BFS.
2. Colour each neighbour with the opposite colour.
3. If a neighbour already has the same colour $\Rightarrow$ not bipartite.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

4.13 Problem 15 — Detect Cycle in Directed Graph

P15: Cycle Detection — Directed Graph

Determine if a directed graph has a cycle.

Two visited arrays are needed:

- `vis[]` — globally visited
- `dfsVis[]` — visited in *current* DFS call stack

A cycle exists when we reach a node already in `dfsVis[]`.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

§5 Category 3 — Topological Sort Problems

Category 3: Topological Sort Problems

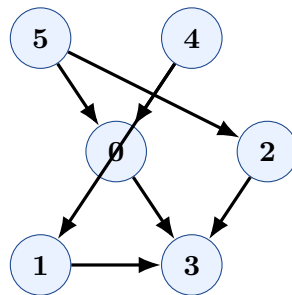
Applicable only to **DAGs** (Directed Acyclic Graphs). Two methods: DFS-based (post-order stack) and BFS-based (Kahn's algorithm using indegrees).

5.1 Problem 16 — Topological Sorting (DFS)

P16: Topological Sort — DFS

For a DAG, produce a linear ordering where every directed edge $u \rightarrow v$ has u before v .

DFS Topo Sort Diagram



Topo Order: $5 \rightarrow 4 \rightarrow 2 \rightarrow 0 \rightarrow 1 \rightarrow 3$

Approach

1. DFS from each unvisited node.
2. After exploring all descendants, push current node to a stack.
3. Pop the stack at end — this is topological order.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

5.2 Problem 17 — Kahn's Algorithm (BFS Topo Sort)

P17: Kahn's Algorithm

Use BFS and indegrees to produce topological order; also detects cycles.

Kahn's Algorithm Walkthrough

Step	Queue	Processed	Indegree Updates
Init	{5,4}	—	all indegrees computed
1	{4,2,0}	5	in[0]−, in[2]−
2	{2,0,1}	4	in[0]−, in[1]−
3	{0,1}	2	in[3]−
4	{1,3}	0	in[3]−
5	{3}	1	in[3]−
6	{}	3	—

Approach

1. Compute indegree of all nodes.
2. Enqueue all nodes with indegree 0.
3. Pop, add to result, decrement neighbours' indegrees; enqueue those reaching 0.
4. If result size $< V \Rightarrow$ cycle exists.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

5.3 Problem 18 — Course Scheduler 1

P18: Course Scheduler 1

Can you complete all `numCourses` given prerequisites? Return `true` or `false`.

Approach

Model as directed graph: edge $b \rightarrow a$ means take b before a . Run Kahn's algorithm; if all nodes are processed (no cycle) return `true`.

Time Complexity: $O(N + E)$

Space Complexity: $O(N + E)$

5.4 Problem 19 — Course Scheduler 2

P19: Course Scheduler 2

Return the topological order of courses to take; empty array if impossible.

Approach

Same as P18; return the BFS order array from Kahn's algorithm. If cycle detected (size $< N$), return empty array.

Time Complexity: $O(N + E)$

Space Complexity: $O(N + E)$

5.5 Problem 20 — Find Eventual Safe States

P20: Find Eventual Safe States

A *safe node* is one from which every path leads to a terminal. Return all safe nodes in sorted order.

Approach — Reverse Graph + Topo Sort

1. Reverse all edges; compute outdegree in original (= indegree in reversed graph).
2. Enqueue terminal nodes (outdegree = 0); run Kahn's on reversed graph.
3. All nodes that get processed are safe.

Time Complexity: $O(N + E)$

Space Complexity: $O(N + E)$

5.6 Problem 21 — Alien Dictionary

P21: Alien Dictionary

Given a sorted alien word list, deduce the character ordering of the alien alphabet using topological sort.

Approach

1. Compare adjacent words in the list; find the first differing character.
2. That gives a directed edge: $c_1 \rightarrow c_2$ (c_1 comes before c_2).
3. Apply Kahn's algorithm on the resulting graph of characters.

Time Complexity: $O(N)$ $N = \text{total characters}$

Space Complexity: $O(K)$ $K = \text{unique chars}$

§6 Category 4 — Shortest Path Problems

Category 4: Shortest Path Algorithms

Covers six distinct algorithms for varying graph types: BFS (unit weight), Topo+DP (DAG), Dijkstra (non-negative weights), Bellman-Ford (negative weights), Floyd-Warshall (all-pairs) and specialised variants.

Algorithm Selection Guide

Scenario	Graph Type	Algorithm	Time
All edges weight 1	Any (undir)	BFS	$O(V+E)$
DAG, any weights	Directed, Acyclic	Topo + DP	$O(V+E)$
No negative weights	Any	Dijkstra	$O(E \log V)$
Negative weights (no neg cycle)	Directed	Bellman-Ford	$O(VE)$
All-pairs shortest path	Any	Floyd-Warshall	$O(V^3)$
At most K stops	Directed	Modified BFS	$O(EK)$

6.1 Problem 22 — Shortest Path (Unit Distance)

P22: Shortest Path — Unit Weights

Find shortest distance from source to all vertices in an unweighted graph.

Approach

Standard BFS; distance of a node = level in BFS tree. Unvisited = -1 .

Time Complexity: $O(N + M)$

Space Complexity: $O(N)$

6.2 Problem 23 — Shortest Path in DAG

P23: Shortest Path in DAG

DAG with weighted edges; find shortest path from source (vertex 0) to all other vertices.

Approach

1. Compute topological order (DFS-based).
2. Initialise $\text{dist}[] = \infty$; $\text{dist}[\text{src}] = 0$.
3. Process each vertex in topo order: for each outgoing edge (u, v, w) update $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w)$.
4. Replace remaining ∞ values with -1 .

Time Complexity: $O(N + M)$

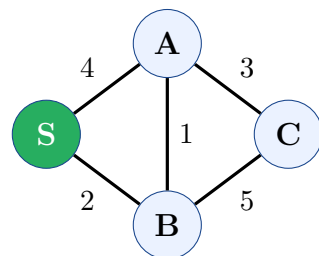
Space Complexity: $O(N + M)$

6.3 Problem 24 — Dijkstra's Algorithm

P24: Dijkstra's Algorithm

Shortest distances from a source vertex in a weighted undirected graph (non-negative weights).

Priority Queue Simulation



Final Distances

Node	dist
S	0
A	3
B	2
C	6

Approach

1. Initialise $\text{dist}[\text{src}] = 0$, all others $= \infty$.
2. Use a min-heap (priority_queue) of $(\text{dist}, \text{node})$.
3. Pop minimum; for each neighbour: if $\text{dist}[u] + w < \text{dist}[v]$, update and push to heap.
4. Continue until heap is empty.

Time Complexity: $O(E \log V)$

Space Complexity: $O(V + E)$

6.4 Problem 25 — Shortest Path in Binary Matrix

P25: Shortest Path in Binary Matrix

Find shortest 8-directional path of 0s from $(0, 0)$ to $(n - 1, n - 1)$.

Approach

BFS from $(0, 0)$; explore all 8 directions; each step increments path length. Return level when reaching $(n - 1, n - 1)$ or -1 if unreachable.

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

6.5 Problem 26 — Path with Minimum Effort

P26: Path with Minimum Effort

Find a path from $(0, 0)$ to $(rows - 1, cols - 1)$ minimising the *maximum* absolute height difference between consecutive cells.

Approach — Modified Dijkstra

1. `effort[]` = ∞ for all except source = 0.
2. Min-heap on `(effort, row, col)`.
3. For each neighbour, new effort = `max(currentEffort, |h1-h2|)`; update if smaller.

Time Complexity: $O(m \times n \times \log(m \times n))$

Space Complexity: $O(m \times n)$

6.6 Problem 27 — Cheapest Flights within K Stops

P27: Cheapest Flights with K Stops

Find cheapest price from `src` to `dst` with at most `k` stops ($= k+1$ edges).

Approach — BFS with Levels

1. Use a queue of `(cost, node)`; process level by level (each level = 1 stop).
2. After at most $k + 1$ levels, `dist[dst]` holds the answer.
3. Unlike Dijkstra, use a simple queue (not min-heap) to enforce exact stop limit.

Time Complexity: $O(E \times K)$
Space Complexity: $O(V)$

6.7 Problem 28 — Network Delay Time

P28: Network Delay Time

Given directed weighted edges, find minimum time for a signal from node k to reach all nodes. Return -1 if impossible.

Approach

Run Dijkstra from node k . Answer = $\max(\text{dist}[])$. If any node is still ∞ , return -1 .

Time Complexity: $O(E \log V)$
Space Complexity: $O(V + E)$

6.8 Problem 29 — Bellman-Ford Algorithm

P29: Bellman-Ford Algorithm

Shortest distances from source; handles negative edges; detects negative cycles.

Key Insight

Relax *all* edges $V - 1$ times. If a further relaxation is possible on the V -th iteration, a **negative cycle** exists.

Bellman-Ford vs Dijkstra

Feature	Dijkstra	Bellman-Ford
Negative Weights	No	Yes
Negative Cycles	Cannot detect	Detects
Time Complexity	$O(E \log V)$	$O(VE)$
Algorithm Type	Greedy (Heap)	DP (relaxation)
Graph Type	Undirected/Directed	Directed only

Time Complexity: $O(V \times E)$
Space Complexity: $O(V)$

6.9 Problem 30 — Floyd-Warshall Algorithm

P30: Floyd-Warshall

Find shortest paths between *every pair* of vertices in a weighted directed graph.

DP Recurrence

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j]) \quad \forall k \in [0, V)$$

Approach

1. Initialise `dist[i][j]` from adjacency matrix (`-1` becomes ∞ , diagonal = 0).
2. Triple loop over intermediate vertex k , source i , target j ; apply recurrence.
3. After V iterations, all pairs are optimal.

Time Complexity: $O(V^3)$

Space Complexity: $O(V^2)$

6.10 Problem 31 — City with Smallest Number of Neighbours

P31: City with Min Reachable Cities

Find the city reachable to the fewest other cities within `distanceThreshold`. Ties broken by largest city number.

Approach

Run Floyd-Warshall; then for each city count reachable cities within threshold; return city with minimum count (largest index on tie).

Time Complexity: $O(V^3)$

Space Complexity: $O(V^2)$

6.11 Problem 32 — Number of Ways to Arrive at Destination

P32: Count Shortest Paths

Count distinct ways to travel from node 0 to node $n - 1$ in minimum time.

Approach — Dijkstra + Count Array

1. Run Dijkstra; maintain `ways[v]` = number of shortest paths to v .
2. If shorter path found: `ways[v] = ways[u]`.
3. If equal path found: `ways[v] += ways[u]`.
4. Return `ways[n-1] mod (1e9+7)`.

Time Complexity: $O(E \log V)$

Space Complexity: $O(V)$

§7 Category 5 — Minimum Spanning Tree & Disjoint Set Union

Category 5: MST & DSU

A **Minimum Spanning Tree** connects all vertices with minimum total edge weight. Two algorithms: Prim's (greedy BFS-like) and Kruskal's (sort edges + DSU). The **Disjoint Set Union (DSU)** data structure supports efficient union-find operations, enabling several classic problems.

Prim's vs Kruskal's

Feature	Prim's	Kruskal's
Approach	Grow from one vertex	Sort all edges globally
Data Structure	Priority Queue	Disjoint Set Union
Best For	Dense graphs	Sparse graphs
Time Complexity	$O(E \log V)$	$O(E \log E)$
Handles Disconnected	No (one component)	Yes (forest of MSTs)

7.1 Problem 33 — Prim's Algorithm

P33: Prim's Algorithm — MST Sum

Find the sum of weights of MST edges.

Approach

1. Start with vertex 0; min-heap of (weight, node).
2. Pop min; if not yet in MST, add weight to sum, mark in MST.
3. Push all edges from new MST node to heap.
4. Repeat until V nodes are in MST.

Time Complexity: $O(E \log V)$

Space Complexity: $O(V + E)$

7.2 Problem 34 — Kruskal's Algorithm

P34: Kruskal's Algorithm — MST Sum

Find the sum of weights of MST edges using edge sorting and DSU.

Approach

1. Sort all edges by weight.
2. For each edge (u, v, w) : if $\text{find}(u) \neq \text{find}(v)$, include in MST and $\text{union}(u, v)$; add w to sum.
3. Stop when $V - 1$ edges are included.

Time Complexity: $O(E \log E)$

Space Complexity: $O(V)$

7.3 Problem 35 — Number of Operations to Make Network Connected

P35: Network Connectivity via DSU

Minimum cables to reconnect all computers. Return -1 if cables are insufficient.

Approach

If $|\text{connections}| < n-1$: impossible, return -1 . Use DSU to count connected components c ; answer $= c - 1$.

Time Complexity: $O(n \cdot \alpha(n))$

Space Complexity: $O(n)$

7.4 Problem 36 — Most Stones Removed

P36: Most Stones Removed

Remove maximum stones from a 2-D plane; a stone can be removed if it shares a row or column with another un-removed stone.

Key Observation

The maximum removable stones $=$ total stones $-$ number of connected components. Two stones are connected if they share a row *or* column. Use DSU to union stones sharing a row/column.

Time Complexity: $O(n \cdot \alpha(n))$

Space Complexity: $O(n)$

7.5 Problem 37 — Account Merge

P37: Account Merge

Merge accounts sharing a common email; return sorted merged accounts.

Approach

1. Map each email to an account index.
2. Union accounts sharing an email via DSU.
3. Group emails by their DSU root; sort each group.
4. Prepend account name to each group.

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

7.6 Problem 38 — Number of Islands 2 (Online DSU)

P38: Number of Islands 2

Start with an all-water grid; add land cells one at a time; after each operation output the current island count.

Approach

For each land addition at (r, c) : add a new component; check 4 neighbours already land; union with them (decrement count for each union). Append count to result.

Time Complexity: $O(k \cdot \alpha(nm))$

Space Complexity: $O(n \times m)$

7.7 Problem 39 — Making Large Island

P39: Making Large Island

Flip one 0 to 1; return the largest possible island.

Approach

1. Build DSU for all existing 1 cells.
2. For each 0 cell, find distinct island roots in its 4 neighbours; sum their sizes + 1 (for the flipped cell).

3. Answer = max over all 0 cells (also check if no 0 exists).

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

7.8 Problem 40 — Swim in Rising Water

P40: Swim in Rising Water

Find the minimum time t to swim from $(0, 0)$ to $(n - 1, n - 1)$; at time t you can only pass through cells with elevation $\leq t$.

Approach — Modified Dijkstra (Min-Max)

1. Min-heap of (elevation, row, col).
2. Track t = running max elevation encountered.
3. When $(n - 1, n - 1)$ is popped, return t .

Time Complexity: $O(n^2 \log n)$

Space Complexity: $O(n^2)$

§8 Category 6 — Advanced Graph Algorithms

Category 6: Advanced Algorithms

Two classical advanced algorithms: **Tarjan's Bridge Finding** (critical connections) and **Kosaraju's SCC** algorithm.

8.1 Problem 41 — Bridges in a Graph (Tarjan's Algorithm)

P41: Critical Connections / Bridges

Find all edges whose removal disconnects the graph.

Two Arrays in Tarjan's Algorithm

Array	Meaning	Initialisation
<code>tin[u]</code>	Discovery time when node u is first visited	Set at DFS entry time
<code>low[u]</code>	Minimum discovery time reachable from subtree of u	Initially = <code>tin[u]</code> ; updated via back/tree edges

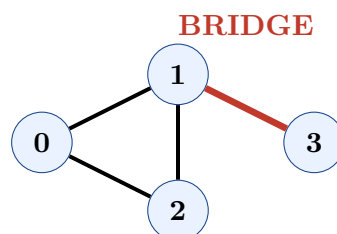
Bridge Condition

Edge (u, v) is a **bridge** if and only if:

$$\text{low}[v] > \text{tin}[u]$$

This means: no back-edge from v 's subtree can reach u or above.

Visualisation



$$\text{low}[3] > \text{tin}[1] \Rightarrow \text{bridge}$$

Approach

1. DFS with a global timer; set $\text{tin}[u] = \text{low}[u] = \text{timer}++$ on entry.
2. For each neighbour v : if unvisited, DFS into it, then $\text{low}[u] = \min(\text{low}[u], \text{low}[v])$; check bridge condition.
3. If v is visited and not parent: $\text{low}[u] = \min(\text{low}[u], \text{tin}[v])$.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

8.2 Problem 42 — Strongly Connected Components (Kosaraju's Algorithm)

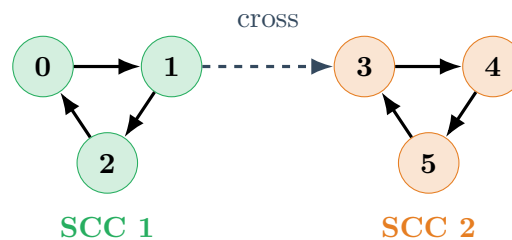
P42: Strongly Connected Components

Count (and identify) SCCs in a directed graph. A SCC is a maximal set of vertices such that there is a path between every pair.

Kosaraju's Three Steps



SCC Visualisation



Approach

1. DFS on original graph; push each node to stack on completion.
2. Reverse all edges (transpose).
3. Pop from stack; DFS on transposed graph — each DFS tree = one SCC.
4. Count the number of DFS calls in step 3.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

§9 Complete Complexity Summary Table

#	Problem	Category	Time	Space
1	Count Graphs	Fundamentals	$O(1)$	$O(1)$
2	Graph Representation	Fundamentals	$O(m)$	$O(n+2m)$
3	BFS	Traversal	$O(V+E)$	$O(V)$
4	DFS	Traversal	$O(V+E)$	$O(V)$
5	Count Provinces	Traversal	$O(n^2)$	$O(n)$
6	Rotten Oranges	Traversal	$O(m*n)$	$O(m*n)$
7	Flood Fill	Traversal	$O(m*n)$	$O(m*n)$
8	Cycle Undirected	Traversal	$O(V+E)$	$O(V)$
9	01 Matrix	Traversal	$O(m*n)$	$O(m*n)$
10	Surrounded Regions	Traversal	$O(m*n)$	$O(m*n)$
11	Enclaves	Traversal	$O(m*n)$	$O(m*n)$
12	Word Ladder	Traversal	$O(n^2 * m)$	$O(n^2)$
13	Distinct Islands	Traversal	$O(n*m)$	$O(n*m)$
14	Bipartite Graph	Traversal	$O(V+E)$	$O(V)$
15	Cycle Directed	Traversal	$O(V+E)$	$O(V)$
16	Topological Sort (DFS)	Topo Sort	$O(V+E)$	$O(V)$
17	Kahn's Algorithm	Topo Sort	$O(V+E)$	$O(V)$
18	Course Scheduler 1	Topo Sort	$O(N+E)$	$O(N+E)$
19	Course Scheduler 2	Topo Sort	$O(N+E)$	$O(N+E)$
20	Find Eventual Safe	Topo Sort	$O(N+E)$	$O(N+E)$
21	Alien Dictionary	Topo Sort	$O(N)$	$O(K)$
22	SP Unit Distance	Shortest Path	$O(N+M)$	$O(N)$
23	SP in DAG	Shortest Path	$O(N+M)$	$O(N+M)$
24	Dijkstra's Algorithm	Shortest Path	$O(E \log V)$	$O(V+E)$
25	SP Binary Matrix	Shortest Path	$O(n^2)$	$O(n^2)$
26	Min Effort Path	Shortest Path	$O(mn \log mn)$	$O(mn)$
27	Cheapest K Stops	Shortest Path	$O(E*K)$	$O(V)$
28	Network Delay Time	Shortest Path	$O(E \log V)$	$O(V+E)$
29	Bellman-Ford	Shortest Path	$O(V*E)$	$O(V)$
30	Floyd-Warshall	Shortest Path	$O(V^3)$	$O(V^2)$
31	City Min Neighbours	Shortest Path	$O(V^3)$	$O(V^2)$
32	Ways to Destination	Shortest Path	$O(E \log V)$	$O(V)$

#	Problem	Category	Time	Space
33	Prim's Algorithm	MST	$O(E \log V)$	$O(V+E)$
34	Kruskal's Algorithm	MST	$O(E \log E)$	$O(V)$
35	Network Operations	MST/DSU	$O(n \cdot \alpha(n))$	$O(n)$
36	Most Stones Removed	MST/DSU	$O(n \cdot \alpha(n))$	$O(n)$
37	Account Merge	MST/DSU	$O(n \log n)$	$O(n)$
38	Number of Islands 2	MST/DSU	$O(k \cdot \alpha(nm))$	$O(n \cdot m)$
39	Making Large Island	MST/DSU	$O(n^2)$	$O(n^2)$
40	Swim Rising Water	MST	$O(n^2 \log n)$	$O(n^2)$
41	Bridges (Tarjan)	Advanced	$O(V+E)$	$O(V)$
42	SCCs (Kosaraju)	Advanced	$O(V+E)$	$O(V)$

§10 Algorithm Patterns — Quick Reference

BFS Pattern

```

1 queue<int> q;
2 vector<bool> vis(n, false);
3 q.push(src); vis[src] = true;
4 while (!q.empty()) {
5     int u = q.front(); q.pop();
6     ans.push_back(u);
7     for (int v : adj[u])
8         if (!vis[v]) {
9             vis[v] = true;
10            q.push(v);
11        }
12 }

```

Dijkstra Pattern

```

1 priority_queue<pair<int,int>,
2     vector<pair<int,int>>,
3     greater<>> pq;
4 dist[src] = 0; pq.push({0,
5     src});
6 while (!pq.empty()) {
7     auto [d, u] = pq.top();
8     pq.pop();
9     if (d > dist[u]) continue;
10    for (auto [v, w] : adj[u])
11        if (dist[u]+w < dist[v]) {
12            dist[v] = dist[u]+w;
13            pq.push({dist[v], v});
14        }
15 }

```

DFS Pattern

```

1 void dfs(int u, vector<bool>&
2     vis,
3     vector<vector<int>>&
4     adj) {
5     vis[u] = true;
6     ans.push_back(u);
7     for (int v : adj[u])
8         if (!vis[v])
9             dfs(v, vis, adj);
10 }

```

Kahn's (Topo BFS)

```

1 queue<int> q;
2 for (int i=0;i<n;i++)
3     if (indeg[i]==0) q.push(i);
4 while (!q.empty()) {
5     int u = q.front(); q.pop();
6     topo.push_back(u);
7     for (int v : adj[u])
8         if (--indeg[v]==0)
9             q.push(v);
10 }
11 // cycle if topo.size() < n

```

DSU (Union-Find)

```

1 int par[N], rnk[N];
2 void init(int n) {
3     for(int i=0;i<n;i++)
4         par[i]=i, rnk[i]=0;
5 }
6 int find(int x) {
7     if (par[x]!=x)
8         par[x]=find(par[x]); //
9         path compress
10    return par[x];
11 }
12 void unite(int x, int y) {
13     x=find(x); y=find(y);
14     if (x==y) return;
15 }

```

```

14     if (rnk[x]<rnk[y]) swap(x,y);
15     par[y]=x;
16     if (rnk[x]==rnk[y]) rnk[x]++;
17 }

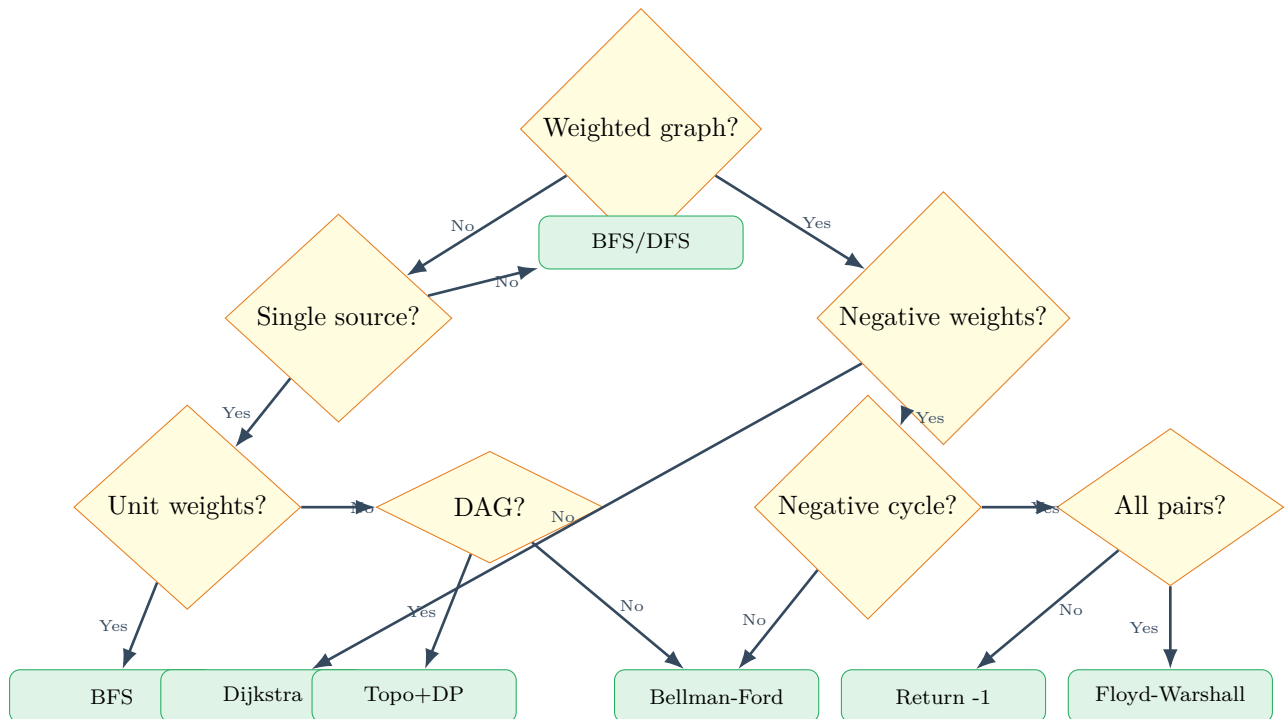
```

Bellman-Ford

```
1 vector<int> dist(V, INT_MAX);  
2 dist[src] = 0;  
3 for (int i=0; i<V-1; i++)  
4     for (auto [u,v,w] : edges)  
5         if (dist[u]!=INT_MAX &&  
6             dist[u]+w < dist[v])  
7             dist[v] = dist[u]+w;  
8 // Check negative cycle:
```

```
9 for (auto [u,v,w] : edges)  
10     if (dist[u]+w < dist[v])  
11         return {-1}; // neg cycle
```

§11 Algorithm Decision Tree



Remember: $\alpha(n)$ (inverse Ackermann) is effectively $O(1)$ for all practical inputs — DSU operations are nearly constant time. $E \log V \approx E \log E$ for sparse graphs since $E = O(V^2)$.